

PROMPT B — DELTA HANDOFF

Status: Verified & Audited

Previous Context: Handoff Packet v1

Focus: New findings, corrections, and architectural pivots based on deep-dive verification.

1. New Verified Facts (The "Delta")

- **Subagent "Deny" Lists are Real:** Contrary to the initial assumption that subagents required tedious allowlists, **Claude Code v2.0.31+** introduced the `disallowedTools` configuration field. You can now define a "Researcher" agent that inherits *all* read tools but explicitly blocks Edit, Write, and Bash.
- **Beads Architecture is Hybrid:** `steveyegge/beads` is not just a markdown list manager. It uses a **local SQLite database** backed by human-readable JSONL files. This allows for high-performance querying (bd ready) while maintaining git-mergeability via hash-based IDs (e.g., `bd-a1b2`) to prevent collisions.
- **Viwo Configuration is YAML-based:** The exact mechanism for configuring a viwo sandbox (Docker) environment is a `viwo.yml` file in the project root. It specifically uses a `postInstall` array to handle dependencies before the agent starts.
- **Hook "Sentinel" Pattern:** Attempting to write complex regex logic (like blocking API keys) directly inside `settings.json` is a verified anti-pattern due to JSON escaping issues. The verified best practice is pointing the hook to an external script (e.g., `.claude/hooks/sentinel.py`).
- **CCUsage is Passive:** `ryoppippi/ccusage` is strictly a CLI reporting tool. It does **not** have a background daemon for active alerting. To get "cost alerts," you must wrap it in a `PreToolUse` hook or shell alias that checks the cost before running commands.

2. New Community Tactics

- **The "Stealth Mode" Bead:** You can run `bd init --stealth` to use Beads for personal task tracking in a shared repository without committing the database or JSONL files to the team's git history.
- **Hyper-Specific "Patrol" Hooks:** Instead of just blocking `rm -rf`, community patterns now use `PreToolUse` hooks to enforce specific "No Vibe" rules, such as preventing commits to

main or blocking the addition of todo comments without an accompanying issue ID.

- **Parallel Worktrees (CCPM):** The automazeio/ccpm tool codifies the "Parallel Agent" tactic by automating the creation of git worktrees linked to GitHub Issues, allowing you to run a "Swarm" of agents on different features simultaneously without file conflicts.

3. Copy/Paste Snippets Discovered

A) Validated viwo.yml (Sandboxing)

Place in project root to ensure the Docker container is ready for autonomy:

YAML

```
# viwo.yml
postInstall:
  - npm install
  - cp.env.example.env
  - npm run build
```

(Source:)

B) The "Deny-by-Default" Subagent

The corrected schema for a safe Researcher agent:

YAML

```
---
name: researcher
description: Analyzes code and docs. READ-ONLY.
tools:
  - Read
  - Grep
  - Glob
  - LS
```

disallowedTools:

- Edit
- Write
- Bash

model: sonnet

(Source:)

4. Contradictions Resolved & Why

- **Conflict:** *Handoff Packet* claimed we need to "allowlist" every tool for safety.
 - **Resolution: False.** The `disallowedTools` field allows for a "Negative Security Model" (Denylist). This is safer and easier to maintain because it automatically allows new, safe read-only tools while permanently blocking dangerous ones.
- **Conflict:** *Community blogs* suggested using `sed` to edit `settings.json`.
 - **Resolution: Dangerous.** `sed` often corrupts the JSON structure. The verified path uses `jq` for surgical insertion of hooks, or manual editing of the file.
- **Conflict:** *Beads* vs. *GitHub Issues*.
 - **Resolution:** They are orthogonal. *Beads* is for *local, high-frequency* agent "thoughts" and task decomposition. *GitHub Issues* (via `ccpm`) are for *team-level* deliverables. Use *Beads* for the agent's "working memory" and *GitHub* for the "committed plan".

5. Still Unclear / Version-Sensitive Items

- **CCPM Dependency:** `ccpm` relies on `gh-sub-issue`, which is a specific GitHub CLI extension. Its compatibility with all OS versions needs verification.
- **VoiceMode Local Config:** The exact environment variable to *force* local Whisper over OpenAI varies by version (`STT_BASE_URL` vs `VOICEMODE_WHISPER_MODEL`). Documentation is inconsistent.
- **Plugin "Rule" Distribution:** The everything-claude-code plugin cannot currently distribute `.claude/rules` files automatically; they must be copied manually. This is a platform limitation.

6. Updated Top 10 Sources

1. (<https://github.com/steveyegge/beads>) (Primary Source for Memory)

2. (<https://github.com/obra/superpowers>) (Primary Source for TDD/Process)
3. [Anthropic Official Hooks Guide](#) (Source of Truth for Syntax)
4. [OverseedAI Viwo](#) (Docker Sandboxing)
5. (<https://github.com/ryoppippi/ccusage>) (Cost Tracking)
6. [AutomazelO CCPM](#) (Project Management/Worktrees)
7. [Everything Claude Code](#) (Config Collection)
8. (<https://github.com/trailofbits/skills>) (Security Auditing)
9. [VoiceMode](#) (Voice I/O)
10. [Claude Code Changelog](#) (Vital for new features like disallowedTools)

7. JSON Appendix

JSON

```
{  
  "new_facts":,  
  "new_tools":,  
  "resolved_conflicts":,  
  "remaining_questions":  
}
```

Things to Verify Next

1. **Windows Compatibility:** Does ccpm's install script (iwr) actually handle the gh-sub-issue dependency correctly on PowerShell?
2. **VoiceMode Offline:** Verify the exact flag or env var to ensure zero data is sent to OpenAI when using VoiceMode.
3. **Beads Sync:** Test if bd sync handles merge conflicts in issues.jsonl gracefully when two agents edit different tasks simultaneously.
4. **Viwo Networking:** Can we lock down the Docker container's network access using viwo.yml or standard Docker compose rules?
5. **Superpowers Integration:** Does the /superpowers:write-plan command integrate with beads tasks, or do they fight for control of the "plan"?
6. **Sentinel Latency:** Does adding a Python script to PreToolUse introduce noticeable lag on every command?
7. **Skill Updates:** Do trailofbits/skills auto-update, or do we need to manually pull new vulnerability checks?
8. **Cost Alerts:** Can we write a simple 5-line bash wrapper for ccusage that blocks execution if daily spend > \$10?